Global Seismic Trends

Data-Driven Earthquake Insights

Project Title	Global Seismic Trends: Data-Driven Earthquake Insights
Domain	Disaster Management · Geoscience · Seismology
Skills	Python · Pandas · Regex · PostgreSQL · SQL · Streamlit
Data Source	USGS Earthquake API (minMagnitude ≥ 3, last 5 years)
Author	Shreejeevan Chinnadurai
GitHub	github.com/Shreejeevanchinnadurai
LinkedIn	linkedin.com/in/shreejeevan-chinnadurai
Portfolio	shreejeevanchinnadurai.github.io/portfolio

1. Problem Statement

- Analyze and interpret global earthquake data to identify seismic patterns, trends, and risk zones.
- Build a data-driven system using API-based retrieval, preprocessing, and SQL analytics for meaningful earthquake insights.

2. Business Use Cases

- Enable governments, insurers, and researchers to assess earthquake risks, plan disaster management strategies, and support informed decision-making.
- Facilitate data-driven policies for urban safety, infrastructure resilience, and effective emergency response.

3. Project Approach

- **Data Collection:** Retrieved earthquake data from the USGS API (minMagnitude >= 3) by looping month-by-month over the last 5 years. Extracted properties and geometry fields. Converted timestamps from milliseconds to datetime.
- **Data Cleaning:** Removed duplicates. Normalized text fields to lowercase. Cleaned the place column by removing noise suffixes. Filled missing values: dmin, rms, gap, cdi, mmi with median; nst, tsunami, felt with 0.
- **Feature Engineering:** Extracted year, month, day, day_of_week from time. Used str.rsplit(',') to derive region and country from place. Created depth_flag (Shallow/Deep) and mag_flag (Weak/Strong/Destructive).
- **Database Storage:** Loaded the cleaned 34-column DataFrame into PostgreSQL (globalsesmic_db) using SQLAlchemy's to_sql() with if_exists='replace'.
- **SQL Analysis:** Wrote 30 analytical SQL queries across 6 categories covering magnitude, time, alerts, event types, seismic patterns, and depth analysis.
- **Dashboard:** Built an interactive Streamlit dashboard with KPI metrics, 30-query dropdown, dataframe display, and dynamic charts (bar, line, area).

4. Data Collection — USGS API

API Endpoint: https://earthquake.usgs.gov/fdsnws/event/1/query

Parameters: format=geojson, starttime/endtime (monthly loop), minmagnitude=3

Code Snippet

```
for year in range(start_year, end_year + 1):
    for month in range(1, 13):
        params = {"format":"geojson","starttime":start_date,
                  "endtime":end_date,"minmagnitude":3}
        response = requests.get(url, params=params)
        for f in data['features']:
            p = f['properties'];  g = f['geometry']['coordinates']
            records.append({"time": pd.to_datetime(p.get("time"), unit="ms"),
                            "latitude": g[1], "longitude": g[0], "depth_km": g[2], ...})
```

5. Data Cleaning Steps

- **Duplicates:** df.drop_duplicates(inplace=True)
- **Text normalization:** alert, magType, status, type, net, sources, types → .str.lower().str.strip()
- **Place cleaning:** Removed ' region', ' earthquakes', ' earthquake sequence' suffixes from place column
- **Region/Country extraction:** df['region'] = place.str.rsplit(',',n=1).str[0] | df['country'] = place.str.rsplit(',',n=1).str[-1]
- **Numeric conversion:** mag, depth_km, nst, dmin, rms, gap → pd.to_numeric(..., errors='coerce')
- **Missing values (median):** dmin, rms, gap, cdi, mmi filled with column median
- **Missing values (zero):** nst, tsunami, felt filled with 0

6. Feature Engineering

```
df['year']        = df['time'].dt.year
df['month']       = df['time'].dt.month_name()
df['day']         = df['time'].dt.day
df['day_of_week'] = df['time'].dt.day_name()
df['region']      = df['place'].str.rsplit(',', n=1).str[0]
df['country']     = df['place'].str.rsplit(',', n=1).str[-1]
df['depth_flag']  = df['depth_km'].apply(lambda x: 'Shallow' if x < 70 else 'Deep')
df['mag_flag']    = df['mag'].apply(lambda x: 'Weak' if x < 4 else 'Strong' if x < 6 else
'Destructive')
```

7. Dataset Features — 34 Columns

Original Column (26) | *** Derived / Extracted Column (8)**

#	Feature	Type	Description
1	**id**	*String*	Unique identifier for each earthquake event. Primary key.
2	**time**	*Datetime*	Exact timestamp of occurrence — converted from milliseconds.
3	**updated**	*Datetime*	Most recent update timestamp for the record.
4	**latitude**	*Float*	Epicenter latitude coordinate.
5	**longitude**	*Float*	Epicenter longitude coordinate.
6	**depth_km**	*Float*	Depth of earthquake in kilometers.
7	**mag**	*Float*	Magnitude — primary strength metric.
8	**magType**	*String*	Measurement method: mb, ml, mww, etc.
9	**place**	*String*	Location description — cleaned of suffix noise.
10	**status**	*String*	reviewed (high confidence) or automatic.
11	**tsunami**	*Integer*	Tsunami generated: 1=yes, 0=no.
12	**alert**	*String*	PAGER alert level: green, yellow, orange, red.

13	**felt**	*Integer*	Number of people who reported feeling the earthquake.
14	**cdi**	*Float*	Community Decimal Intensity. Nulls filled with median.
15	**mmi**	*Float*	Modified Mercalli Intensity. Nulls filled with median.
16	**sig**	*Integer*	Significance score combining magnitude, felt, and impact.
17	**net**	*String*	Network that authored the preferred event solution.
18	**code**	*String*	Identifier code assigned by the reporting network.
19	**ids**	*String*	All event IDs associated with this event.
20	**sources**	*String*	Contributing data sources/networks.
21	**types**	*String*	Product types available for this event.
22	**nst**	*Integer*	Seismic stations used for location. Nulls filled with 0.
23	**dmin**	*Float*	Min distance to nearest station (degrees). Nulls → median.
24	**rms**	*Float*	Root mean square travel time residual. Nulls → median.
25	**gap**	*Float*	Largest azimuthal gap between stations. Nulls → median.
26	**type**	*String*	Event type: earthquake, quarry blast, explosion, etc.
27	**region ***	*String*	Extracted from place using str.rsplit(',') — sub-region.
28	**country ***	*String*	Extracted from place using str.rsplit(',') — country name.
29	**year ***	*Integer*	Extracted from time column.
30	**month ***	*String*	Month name extracted from time (e.g. January).
31	**day ***	*Integer*	Day of month extracted from time.
32	**day_of_week ***	*String*	Day name extracted from time (e.g. Monday).
33	**depth_flag ***	*String*	Shallow (depth_km < 70) or Deep (depth_km >= 70).
34	**mag_flag ***	*String*	Weak (mag<4) / Strong (4–5.9) / Destructive (mag>=6).

8. SQL Analytical Queries — 30 Total

Magnitude & Depth

#	Question	SQL Summary
1	**Top 10 Strongest Earthquakes**	`SELECT place, mag FROM earthquakes ORDER BY mag DESC LIMIT 10`
2	**Top 10 Deepest Earthquakes (mag > 7.0)**	`SELECT place, depth_km FROM earthquakes WHERE mag > 7.0 ORDER BY depth_km DESC LIMIT 10`
3	**Shallow Earthquakes (depth < 50 km AND mag > 7.5)**	`SELECT depth_km, place, mag FROM earthquakes WHERE depth_km < 50 AND mag > 7.5 ORDER BY mag DESC`
5	**Average Magnitude per magType**	`SELECT magType, AVG(mag) AS avg_magnitude FROM earthquakes GROUP BY magType ORDER BY avg_magnitude DESC`

Time Analysis

#	Question	SQL Summary
6	Year with Most Earthquakes	`SELECT year, COUNT(*) AS total FROM earthquakes GROUP BY year ORDER BY year DESC`
7	Month with Highest Earthquakes	`SELECT month, COUNT(*) AS total FROM earthquakes GROUP BY month ORDER BY total DESC`
8	Day of Week with Most Earthquakes	`SELECT day_of_week, COUNT(*) FROM earthquakes GROUP BY day_of_week ORDER BY total DESC`
9	Earthquakes per Hour of Day	`SELECT EXTRACT(HOUR FROM time::TIMESTAMP) AS hour, COUNT(*) FROM earthquakes GROUP BY hour ORDER BY hour`

Casualties & Alerts

#	Question	SQL Summary
10	Most Active Reporting Network	`SELECT net, COUNT(*) AS total FROM earthquakes GROUP BY net ORDER BY total DESC LIMIT 10`
11	Top 5 Places by Felt Reports	`SELECT country, sig, felt, mag, depth_km FROM earthquakes ORDER BY felt DESC LIMIT 5`
13	Alert Level Distribution	`SELECT alert, COUNT(*) AS count FROM earthquakes GROUP BY alert`
19	Tsunamis Triggered per Year	`SELECT year, tsunami, COUNT(*) FROM earthquakes GROUP BY year, tsunami ORDER BY year DESC`
20	Earthquakes by Alert Level	`SELECT alert, COUNT(*) AS total FROM earthquakes GROUP BY alert ORDER BY alert DESC`

Event Type & Quality

#	Question	SQL Summary
14	Reviewed vs Automatic Status	`SELECT status, COUNT(*) FROM earthquakes GROUP BY status`
15	Count by Earthquake Type	`SELECT type, COUNT(*) FROM earthquakes GROUP BY type`
16	Count by Data Types Field	`SELECT types, COUNT(*) FROM earthquakes GROUP BY types`
18	Events with High Station Coverage	`SELECT place, mag, nst FROM earthquakes WHERE nst IS NOT NULL ORDER BY nst DESC LIMIT 10`
28	Lowest Data Reliability Events	`SELECT place, gap, rms, mag FROM earthquakes WHERE gap IS NOT NULL AND rms IS NOT NULL ORDER BY gap DESC, rms DESC LIMIT 10`

Seismic Patterns & Trends

#	Question	SQL Summary
21	**Top 5 Countries by Avg Magnitude**	`SELECT country, AVG(mag) AS avg_mag FROM earthquakes GROUP BY country ORDER BY avg_mag DESC LIMIT 5`
22	**Shallow & Deep Earthquakes in Same Month**	`SELECT year, month, COUNT(CASE WHEN depth_flag='Shallow' THEN 1 END) AS shallow, COUNT(CASE WHEN depth_flag='Deep' THEN 1 END) AS ...`
23	**Year-over-Year Earthquake Growth**	`SELECT year, COUNT(*) AS total FROM earthquakes GROUP BY year ORDER BY year`
24	**Top 10 Most Seismically Active Regions**	`SELECT place, COUNT(*) AS frequency, ROUND(AVG(mag)::numeric,2) AS avg_magnitude FROM earthquakes GROUP BY place ORDER BY frequenc...`
27	**Magnitude Diff: Tsunami vs Non-Tsunami**	`SELECT ROUND(AVG(CASE WHEN tsunami=1 THEN mag END)::NUMERIC,2) AS with_tsunami, ROUND(AVG(CASE WHEN tsunami=0 THEN mag END)::NUMER...`

Depth & Location Analysis

#	Question	SQL Summary
25	**Avg Depth Near Equator (lat between -5 and 5)**	`SELECT country, AVG(depth_km) AS avg_depth, COUNT(*) FROM earthquakes WHERE latitude BETWEEN -5 AND 5 GROUP BY country ORDER BY av...`
26	**Highest Shallow-to-Deep Ratio by Country**	`SELECT country, COUNT(CASE WHEN depth_flag='Shallow' THEN 1 END) AS shallow, COUNT(CASE WHEN depth_flag='Deep' THEN 1 END) AS deep...`
30	**Deep-Focus Earthquake Regions (depth > 300 km)**	`SELECT place, COUNT(*) AS total, AVG(depth_km) AS avg_depth, MAX(depth_km) AS max_depth, AVG(mag) AS avg_mag FROM earthquakes WHER...`

9. Streamlit Dashboard

The dashboard connects to PostgreSQL (globalsesmic_db) via SQLAlchemy and displays:

- KPI Metrics row: Total Earthquakes, Average Magnitude, Maximum Magnitude, Countries Affected
- Dropdown selector for all 30 analytical questions
- Dynamic dataframe display (use_container_width=True) for each query result
- Automatic chart rendering: bar chart (Q5,8,9,10,20), line chart (Q6,23), area chart (Q19)
- Custom CSS: dark gradient background (#0f172a to #1e293b), styled metric cards
- Queries not supported by dataset (Q4, Q12, Q17, Q29) display a descriptive info message

10. Results

- Clean, normalized earthquake dataset covering 5 years with 34 features (26 original + 8 derived).
- PostgreSQL database (globalsesmic_db) loaded via SQLAlchemy with full earthquake records.
- 30 SQL analytical queries across 6 categories providing actionable seismic insights.
- Interactive Streamlit dashboard with KPI cards, dynamic query rendering, and auto-charts.
- Derived depth_flag (Shallow/Deep) and mag_flag (Weak/Strong/Destructive) for richer analysis.

11. Tech Stack

Technology	Version/Tool	Purpose
Python	*3.x*	Core programming language
PostgreSQL	*15+*	Database — globalsesmic_db
Pandas	*Latest*	Data manipulation and preprocessing
Regex / str methods	*Built-in*	Cleaning place, extracting region & country
Streamlit	*Latest*	Interactive web dashboard
Matplotlib	*Latest*	Histogram and chart visualizations
SQLAlchemy	*Latest*	Database connection (psycopg2 driver)
USGS API	*fdsnws/event/1*	Real-time earthquake data retrieval